Agentic LLM-based Security Testing for BOLA in REST APIs

Valérie Jacobs
Supervisors: Prof. Serge Demeyer & Tolgahan Bardakci

Master's Program in Data Science and AI
University of Antwerp

September 27, 2026

1 Problem Statement

Large Language Models (LLMs) have become increasingly more capable of generating high-quality test cases [1], [2], [3]. However, their ability to systematically identify complex authorization vulnerabilities, such as Broken Object Level Authorization (BOLA)/Insecure Direct Object References (IDOR), remains limited. Existing approaches often struggle to reason about dependencies between request parameters and across multiple API interactions. As a result, some approaches rely on broad testing strategies that can generate a large number of invalid requests rather than targeting relevant authorization scenarios [8]. Consequently, many security vulnerabilities are being discovered incidentally, rather than through targeted, context-aware testing [8], [9].

As a result, authorization vulnerabilities such as BOLA/IDOR remain among the most common security vulnerabilites in Representational State Transfer (REST) Application Programming Interfaces (APIs) [5], [9]. This research aims to address this gap, leading to the following research questions:

> RQ1: To what extent can an agentic LLM system improve the discovery of BOLA/IDOR vulnerabilities compared to the state-of-the-art and what are the associated costs?
>
> RQ2: How does the effectiveness of discovered violations change with API complexity?
>
> RQ2.1: What's the relationship between, the OpenAPI documentation quality and the quality of the results?
>
> RQ3: To what extent can the generated test scenarios be executed successfully without manual intervention?

2 Background

Automated security testing of REST APIs can broadly be divided into several approaches, namely Static Application Security Testing (SAST), Dynamic Application Security Testing (DAST), Stateful fuzzing, Formal Mathematical Models, Rule-based API scanners and more recently, LLM-driven test generation [10], [19], [9].

At a high level, these approaches can be divided into static and dynamic analysis. SAST approaches analyze API specifications without executing the application. In the context of BOLA detection, static analysis can identify endpoints and parameters related to object access. For example, Barabanov et al.'s tool annotates each endpoint with 'BOLA properties' and it applies a rule-based attack analyzer to the annotated specification to generate possible attack vectors [11]. However, such approaches often lack the required runtime information to determine whether an object can actually be accessed by an unauthorized user [10]. Similarly, DAST approaches test a running application by sending requests and analyzing its responses. Although these techniques can identify a range of API vulnerabilities, previous research indicates SAST and DAST approaches are not very effective at detecting BOLA vulnerabilities [20].

Stateful fuzzing attempts to address some of the limitations of traditional fuzzing by considering dependencies between API requests. Tools of the RESTler family can generate sequences of requests based on an API's structure [6]. Other approaches, including BACFuzz and BACScan, specifically consider access-control vulnerabilities. However, these techniques generally do not target BOLA and they usually operate under a single identity [19] [7] [21].

Another approach is to model API behaviour and dependencies explicitly. Links2CPN, for example, uses coloured Petri nets to model API interactions and dependencies and uses server logs to support the detection of access-control violations. However, this approach requires JSON-formatted server logs. As no tool was provided to generate them, implementing this functionality manually would be too time-consuming [23]. Which is why Links2CPN could not be included in the experimental evaluation.

More recently, LLMs have been investigated for API security testing. LLMs can generate test cases from API specifications and can use contextual information to construct more targeted scenarios [1], [2], [3]. However, existing LLM-based approaches for BOLA detection remain limited. For example, BOLABuster has been proposed as an LLM-based approach, but its publicly available description provides limited information about its prompt design, model configuration, and empirical evaluation [10]. Other approaches, such as Karate-BOLA-Guard, use Retrieval-Augmented Generation (RAG) to provide an LLM with relevant OpenAPI information and BOLA-specific knowledge [8].

In attempting to reproduce Karate-BOLA-Guard, we encountered several issues that prevented us from obtaining a functioning setup. The project's dependency requirements contained conflicts. We also encountered errors when attempting to store the OpenAPI fragments which is required by the system. Hence, we were unable to obtain a functional implementation of Karate-BOLA-Guard to include it in our experimental evaluation.

Recent rule-based API scanners such as AuthProbe, OWASP API Security Testing Framework (ASTF) and Akto use multiple authenticated identities and compare access to resources owned by different users. AuthProbe, for example, parses an OpenAPI specification, identifies relevant endpoints, discovers resources belonging to different accounts, and tests whether one account can access another account's objects [21]. ASTF similarly uses multiple authentication tokens to determine whether objects accessible to one identity can also be accessed by another [22]. We were able to evaluate these two tools succesfully and they shall be used in the comparison study. Akto also supports BOLA testing by importing an OpenAPI specification and replaying requests while replacing the legitimate user's authentication credentials with those of another user [24]. But, we were not able to evaluate this tool because of an error that kept occuring when trying to start the BOLA test suite on the OpenAPI spec.

Overall static analysis can identify potentially relevant authorization relationships but lacks runtime information, stateful fuzzers can reason about API dependencies but are not specifically designed for BOLA, and rule-based API scanners can directly test authorization boundaries but generally rely on predefined testing strategies. LLM-based approaches offer the potential to generate more context-aware test scenarios, but their ability to produce executable multi-step tests remain less established.

3 Relevancy

REST APIs form the backbone of modern web applications, cloud services, and mobile applications [4]. Ensuring that authorization is correctly enforced is therefore very important. According to the OWASP API Security Top 10, BOLA has consistently been ranked as the most critical API security vulnerability [17], [18], [5].

This research aims to address this limitation by developing an agentic AI system that analyzes an OpenAPI specification and generates security-focused, multi-step authorization tests that reason about multiple users, object ownership, and resource lifecycles. So, instead of executing isolated requests, the proposed system will reason about sequences of API interactions involving multiple identities and object states to uncover authorization flaws. Besides contributing to software testing research, such a system could assist developers in finding security vulnerabilities earlier during development while producing reproducible

and explainable test cases.

4 Why Can I Solve This?

Throughout my studies I have developed an interest in cybersecurity, software testing, and AI. I have practical experience developing REST APIs and software systems, which allows me to understand both the implementation and testing perspectives.

In addition, my academic background has provided me with knowledge of software testing, software quality, and machine learning techniques. This combination makes me well suited to investigate how agentic LLMs can be applied to automated security testing.

The project also aligns with the expertise of the AnSyMo research group, allowing me to build upon existing research on REST API testing and LLM-based amplification and also extending it towards vulnerability-oriented testing.

5 How to Solve the Problem?

The first step is to analyze the API specification to identify resources, relationships, and ownership information. This will be stored in the agent's context window.

The core of the agentic system will be a planner LLM that generates sequences of API requests. Rather than testing single endpoints independently, the planner LLM will reason about object creation, ownership transfer, lifecycle changes, and access attempts performed by different users.

The generated requests will then be executed against the target API, after which the responses will be analyzed to detect potential authorization violations.

Finally, any discovered vulnerabilities will be written to a report.

6 Evaluation

6.1 Evaluation of RQ1: Detection Effectiveness and Cost

The proposed approach will be evaluated using intentionally vulnerable benchmark REST APIs containing known BOLA's. Johansens et al. [10] provides twelve such APIs with documented BOLA vulnerabilities. Of these, eight were successfully deployed and will therefore be used for evaluation, with their documented BOLA vulnerabilities serving as the ground truth.

The effectiveness of the approach will primarily be evaluated in terms of precision and recall.

$$\text{Precision} = \frac{TP}{TP+FP}$$

Where TP denotes correctly identified BOLA vulnerabilities and FP denotes the incorrect ones. This metric has previously been used in the evaluation of LLM-based penetration testing systems such as PentestGPT [10], [16].

Recall measures the proportion of known BOLA vulnerabilities that are successfully detected:

$$\text{Recall} = \frac{\text{Detected BOLA vulnerabilities}}{\text{Total known BOLA vulnerabilities}}$$

Besides detection effectiveness, the computational cost of the approach will be measured. Following previous work such as RESTler and Karate-BOLA-Guard, the following efficiency metrics will be collected:

- Number of API requests required per confirmed vulnerability
- Number of LLM tokens consumed per confirmed vulnerability
- Total wall-clock execution time until a confirmed vulnerability is discovered

6.2 Evaluation of RQ2: The effect of API complexity

The second research question investigates whether the proposed approach remains effective for APIs with varying complexity. Detection performance will be analyzed with respect to API complexity. To evaluate this, two tools are used to characterize the APIs under test: RAMA and OASQuali. RAMA is used to quantify the structural complexity of an API, while OASQuali is used to assess the quality of its OpenAPI specification.

RAMA: The RAMA CLI provides a set of metrics for measuring API complexity automatically.[13], [14].

- **Average Path Length (APL):** Average length of API paths.

  Average path length captures the overall structural depth of the API. Longer paths generally indicate more complex resource hierarchies and potentially more complex authorization relationships.

- **Arguments per Operation (APO):** Average amount of operation arguments for a service.

  Operations requiring more parameters provide more possible combinations of identifiers that an agent must handle when testing authorization.

- **Biggest Root Coverage (BRC):** Measures how much of the API is reachable from its largest root resource.

  A high BRC indicates that a substantial portion of the API is organized around a common resource. So a low BRC, meaning the resource space is more scatterd, can increase the complexity of relationships that an agent must explore.

- **Data Weight (DW):** Represents the complexity of the data types used in input and output messages; the amount/size of data structure. It is computed by counting all path parameters and all parameters in request and response bodies within all operations.

  More complex request and response structures can increase the amount of information the agent must process when constructing authorization tests.

- **Distinct Message Ratio (DMR):** Captures the complexity/cohesion of data types in an interface; it measures the uniqueness of the structures. It is computed by dividing the count of distinct messages that occur only once in the whole API by the number of all messages.

  A higher DMR suggests that the agent will encounter a wider variety of request and response formats, increasing the reasoning effort required.

- **Number of Roots (NOR):** Measures the number of independent root resources in the API. A root resource is any resource that has no incomming links [12] [13].

  A larger number of roots indicates a more fragmented API structure, requiring the agent to explore more resource hierarchies.

- **Longest Path (LP):** Referring to the length of the longest path of resources connected by links. In other words, the path length of a resource is defined by the number of slashes ("/") in its path[12], [13].

  A high maximum depth indicates that at least some operations require the agent to reason across a long sequence of resource relationships. This can make it more difficult to construct valid requests and maintain the necessary context when testing authorization boundaries.

- **Lack of Message-Level Cohesion (LoC_msg):** Measures how dissimilar the input and output data of operations within an interface are. A higher lack of cohesion requires the agent to handle a wider range of request and response structures.

- **Service Interface Data Cohesion (SIDC):** Measures how strongly operations within a service share input parameters and return types. A higher lack of cohesion requires the agent to handle a wider range of input parameter and return type structures.

- **Weighted Service Interface Count (WSIC):** Measures the size of an interface by considering the number of exposed operations. A larger

interface provides the agent with more operations to explore and more potential interactions through which authorization violations may occur.

Thresholds for these metrics were calculated from a dataset of 1,737 API description files [13]. These thresholds can therefore be used as reference points for determining the relative complexity of the APIs included in the evaluation.

OASQuali: API complexity alone does not determine how difficult an API is for an agent to test. The quality and completeness of the OpenAPI specification also influence how much reliable information is available for understanding the API and generating test cases. OASQuali provides a deterministic assessment of OpenAPI documentation quality across five dimensions and 17 concrete evaluations [15]. The Administrative Governance and Operational Readiness dimensions will be ommited because of irrelevance.

- **Structural Integrity:** This dimension consists of *validate-json* and *validate-oas*, which check whether the specification is valid JSON and conforms to the OpenAPI metamodel [15]. This is relevant because the specification serves as the agent's input for understanding the API. Structural errors can prevent reliable parsing reducing the agent's ability to generate valid BOLA/IDOR tests.
- **Evolutionary Maturity:** This dimension consists of *oas-version*, which checks whether the specification uses OAS version 3.0.0 or later [15]. This is relevant because newer OAS versions provide support for modern specification features. A more up-to-date specification can therefore provide the agent with a more consistent representation of the API, whereas older specifications may limit the information available.
- **Semantic Communicability:** This dimension consists of *route-descriptions*, *response-descriptions*, *parameter-descriptions*, *response-examples*, and *parameter-examples* [15]. These metrics measure how much semantic information is available to the agent. Route and parameter descriptions can clarify the purpose of resources and identifiers, while examples provide concrete information about how requests and responses are structured. Poor semantic communicability can therefore increase the amount of effort required from the agent when reasoning about authorization violations.

Overall, OASQuali complements the RAMA metrics by measuring a different property of the API under test. RAMA characterizes the structural and interface complexity of the API, whereas OASQuali characterizes the quality of the machine-readable specification used by the agent.

6.3 Evaluation of RQ3: Executability of the Generated Tests

The extent to which the generated test scenarios can be executed successfully without manual intervention will be evaluated using execution-based metrics.

The executable scenario rate is defined as:

$$\text{Executable scenario rate} = \frac{\text{Executable generated scenarios}}{\text{All generated scenarios}}$$

Which measures how often the generated API interaction sequences can be executed successfully without manual correction.

7 Timeline

- **July-August:** Conduct a literature review on REST API testing, authorization vulnerabilities, OpenAPI specifications, and agentic LLMs. Define the research scope. Test the existing BOLA detection tools.
- **September-December:** Compare the existing BOLA detection tools to each other. Implement the OpenAPI parser and automatically create multiple testing identities.
- **January-March:** Design the initial architecture. Develop the agentic LLM system capable of generating multi-step authorization test scenarios. Integrate the execution framework. Implement vulnerability detection, execute experiments, and compare the proposed approach with existing techniques.
- **April-May:** Analyze the experimental results and write the master's thesis.
- **June:** Finalize the thesis, prepare the presentation, and defend the research.

References

[1] Tasarsu, M., Tokmak, A. V., & Catal, C. (2026). Test case generation using large language models: A systematic literature review. Cluster Computing, 29(4), 16. https://doi.org/10.1007/s10586-026-06021-z

[2] Schäfer, M., Nadi, S., Eghbali, A., & Tip, F. (2024). An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. IEEE Transactions on Software Engineering, 50(1), 85–105. https://doi.org/10.1109/TSE.2023.3334955

[3] Zhang, Q., Fang, C., Gu, S., Shang, Y., Chen, Z., & Xiao, L. (2025). Large Language Models for Unit Testing: A Systematic Literature Review (arXiv:2506.15227). arXiv. https://doi.org/10.48550/arXiv.2506.15227

[4] Peldszus, S., Rutenkolk, J., Heide, M., Sollmann, J., Klatt, B., Köhne, F., & Berger, T. (2026). Developer Perspectives on REST API Usability: A Study of REST API Guidelines (arXiv:2601.16705). arXiv. https://doi.org/10.48550/arXiv.2601.16705

[5] OWASP Top 10:2025. (2025). OWASP Top 10:2025. https://top10.owasp.org/2025/

[6] Atlidakis, V., Godefroid, P., & Polishchuk, M. (2019, May). RESTler: Stateful REST API Fuzzing. 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE). https://doi.org/10.1109/ICSE.2019.00083

[7] Dharmaadi, I. P. A., Alhanahnah, M., Pham, V.-T., Mohsen, F., & Turkmen, F. (2025). BACFuzz: Exposing the Silence on Broken Access Control Vulnerabilities in Web Applications (arXiv:2507.15984). arXiv. https://doi.org/10.48550/arXiv.2507.15984

[8] Marian Pasca, E., Delinschi, D., Erdei, R., & Matei, O. (2025). LLM-Driven, Self-Improving Framework for Security Test Automation: Leveraging Karate DSL for Augmented API Resilience. IEEE Access, 13, 26. https://doi.org/10.1109/ACCESS.2025.3554960

[9] Tanveer, F., Iradat, F., Iqbal, W., & Ahmad, A. (2025). Towards Secure APIs: A Survey on RESTful API Vulnerability Detection. Computers, Materials & Continua, 84(3), 35. https://doi.org/10.32604/cmc.2025.067536

[10] Johansens, E. (2025, July 4). Detecting BOLA Vulnerabilities with Large Language Models. Google Docs. `https://drive.google.com/file/d/1mqfIWQNgHW_VC2thABM2Ei2AlXvw2BNT/view?usp=drivesdk&usp=embed_facebook`

[11] Barabanov, A., Dergunov, D., Makrushin, D., & Teplov, A. (2022). Automatic detection of access control vulnerabilities via API specification processing. Voprosy Kiberbezopasnosti, 22. https://doi.org/10.48550/arXiv.2201.10833

[12] Haupt, F., Leymann, F., & Vukojevic-Haupt, K. (2017). API governance support through the structural analysis of REST APIs. Computer Science - Research and Development, 33(3), 291–303. https://doi.org/10.1007/s00450-017-0384-1

[13] Bogner, J., Wagner, S., & Zimmermann, A. (2020). Collecting Service-Based Maintainability Metrics from RESTful API Descriptions: Static Analysis and Threshold Derivation (Vol. 1269, p. 14). https://doi.org/10.1007/978-3-030-59155-7_16

[14] RESTful Maintainability Assurance. (2023, August 16). Restful-ma/rama-cli. https://github.com/restful-ma/rama-cli (Original work published 2019)

[15] Decrop, A., Vandeloise, M., Heymans, P., & Perrouin, G. (2026). OASQuali: Automated Quality Analysis of OpenAPI Specifications. In A. Mauri, L. Burgueño, & R. Tommasini (Eds.), Web Engineering (p. 16). Springer Nature Switzerland. https://doi.org/10.1007/978-3-032-29372-5_7

[16] Deng, G., Liu, Y., Mayoral-Vilches, V., Liu, P., Li, Y., Xu, Y., Zhang, T., Liu, Y., Pinzger, M., & Rass, S. (2024). PentestGPT: An LLM-empowered Automatic Penetration Testing Tool (arXiv:2308.06782). arXiv. https://doi.org/10.48550/arXiv.2308.06782

[17] OWASP Top 10 API Security Risks – 2019—OWASP API Security Top 10. (2019). https://api-security.owasp.org/editions/2019/en/0x11-t10/

[18] OWASP Top 10 API Security Risks – 2023—OWASP API Security Top 10. (2023). https://api-security.owasp.org/editions/2023/en/0x11-t10/

[19] Liu, F., Zhang, Y., Li, E., Meng, W., Shi, Y., Wang, Q., Wang, C., Lin, Z., & Yang, M. (2025). BACScan: Automatic Black-Box Detection of Broken-Access-Control Vulnerabilities in Web Applications. Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS '25, 1320–1333. https://doi.org/10.1145/3719027.3744825

[20] Mazon, R., & Chen, J. (2024, August 12). Harnessing LLMs for Automating BOLA Detection. Unit 42. https://unit42.paloaltonetworks.com/automated-bola-detection-and-ai/

[21] Barach, J. (2026). AuthProbe: Specification-Driven, Multi-Identity Detection of Broken Object-Level Authorization in Recruitment API (arXiv:2607.20574). arXiv. https://doi.org/10.48550/arXiv.2607.20574

[22] OWASP. (2026, September 20). OWASP/www-project-api-security-testing-framework. https://github.com/OWASP/www-project-api-security-testing-framework (Original work published 2025)

[23] Santos Filho, A., Rodríguez, R. J., & Feitosa, E. L. (2025). Automated broken object-level authorization attack detection in REST APIs through OpenAPI to colored petri nets transformation. International Journal of Information Security, 24(2). https://doi.org/10.1007/s10207-024-00970-5

[24] Akto. (2026). Akto-api-security/akto [Java]. https://github.com/akto-api-security/akto (Original work published 2023)